

Syllabus

Introduction: Algorithm, Performance Analysis-Space Complexity, Time complexity, Asymptotic Notations- Big oh notation, Omega notation, Theta notation and Little oh notation. **Page No.--1**

Recursion: Introduction, Fibonacci sequence, Climbing Stairs, Reverse String, Happy Number, Greatest Common Divisor, Strobogrammatic Number II. **Page No.--15**

Divide and Conquer: General method, Master's Theorem, Quick sort, Merge sort, Applications: Majority Element, Calculate $\text{pow}(x,n)$. **Page No.--37**

Introduction

What is an algorithm?

- An algorithm is a procedure used for solving a problem or performing a computation. Algorithms act as an exact list of instructions that conduct specified actions step by step in either hardware- or software-based routines.
- Algorithms are widely used throughout all areas of IT. In mathematics and computer science, an algorithm usually refers to a small procedure that solves a recurrent problem. Algorithms are also used as specifications for performing data processing and play a major role in automated systems.
- An algorithm could be used for sorting sets of numbers or for more complicated tasks, like recommending user content on social media. Algorithms typically start with initial input and instructions that describe a specific computation. When the computation is executed, the process produces an output.

Qualities(Criteria) of a Good Algorithm

1. **Input:** Zero or more quantities are externally supplied.
2. **Output:** At least one quantity is produced.
3. **Definiteness:** Each instruction is clear and unambiguous of an algorithm.
4. **Finiteness:** If we trace out the instructions of an algorithm for all cases, then the algorithm should terminate after a finite number of steps.
5. **Effectiveness:** Instruction is basic enough to be carried out.

How do algorithms work?

- Algorithms can be expressed as natural languages, programming languages, pseudocode, flowcharts and control tables. Natural language expressions are rare, as they are more ambiguous. Programming languages are normally used for expressing algorithms executed by a computer.
- Algorithms use an initial input along with a set of instructions. The input is the initial data needed to make decisions and can be represented in the form of numbers or words. The input data gets put through a set of instructions, or computations, which can include arithmetic and decision-making processes. The output is the last step in an algorithm and is normally expressed as more data.

What are different types of algorithms?

There are several types of algorithms, all designed to accomplish different tasks. For example, algorithms perform the following:

- **Search engine algorithm.** This algorithm takes strings of keywords and operators as input, searches its associated database for relevant webpages and returns results.
- **Encryption algorithm.** This computing algorithm transforms data according to specified actions to protect it. A symmetric key algorithm, such as the Data_Encryption_Standard, for example, uses the same key to encrypt and decrypt data. As long as the algorithm is sufficiently sophisticated, no one lacking the key can decrypt the data.
- **Greedy algorithm.** This algorithm solves optimization problems by finding the locally optimal solution, hoping it is the optimal solution at the global level. However, it does not guarantee the most optimal solution.
- **Recursive algorithm.** This algorithm calls itself repeatedly until it solves a problem. Recursive algorithms call themselves with a smaller value every time a recursive function is invoked.
- **Backtracking algorithm.** This algorithm finds a solution to a given problem in incremental approaches and solves it one piece at a time.
- **Divide-and-conquer algorithm.** This common algorithm is divided into two parts. One part divides a problem into smaller sub problems. The second part solves these problems and then combines them together to produce a solution.
- **Dynamic programming algorithm.** This algorithm solves problems by dividing them into subproblems. The results are then stored to be applied for future corresponding problems.

- **Brute-force algorithm.** This algorithm iterates all possible solutions to a problem blindly, searching for one or more solutions to a function.
- **Sorting algorithm.** Sorting algorithms are used to rearrange data structure based on a comparison operator, which is used to decide a new order for data.
- **Hashing algorithm.** This algorithm takes data and converts it into a uniform message with a hashing
- **Randomized algorithm.** This algorithm reduces running times and time-based complexities. It uses random elements as part of its logic.

Pseudo code for expressing algorithms

We present the algorithms using pseudo code that looks like C and Pascal code.

1. Comments begin with // and continue until end of the line.
2. Blocks are indicated with braces: { }.
3. i). A compound statement.
- ii). Body of a function.
3. i). The data types of variables are not explicitly declared.
- ii). The types will be clear from the context.
- iii). Whether a variable is global or local to a function will also be clear from the context.
- iv). We assume simple data types such as integer, float, char, boolean, and so on.
- v). Compound data types can be formed with **records**.

```
node = record
{
    datatype_1  data_1;
    :
    datatype_n  data_n;
    node        *link
}
```

data items of a record can be accessed with $\rightarrow$ and period(.)

4. Assignment statement.
- < variable > := < expression >
5. Boolean values are **true** and **false**. Logical operators are **and**, **or** and **not** and the relational operators are <, ≤, =, ≠, ≥ and >.
6. Elements of arrays are accessed using [and]. For example the (i,j)th element of the array A is denoted as A[i,j].
7. The following looping statements are used: for, while, and repeat until.

The general form of a **while** loop:

```
while( condition ) do
{
    statement_1;
    :
    statement_n;
}
```

The general form of a **for** loop:

```
for variable := value1 to value2 step step do
{
    statement_1;
    :
    statement_n;
}
```

- Here value1, value2, and step are arithmetic expressions.
- The clause “**step** step” is optional and taken as +1 if it does not occur.
- *step* could be either positive or negative.

Ex: 1: for i:= 1 to 10 step 2 do // increment by 2, 5 iterations
2: for i:= 1 to 10 do // increment by 1, 10 iterations

while loop as follows:

```
variable:=value1;
incr:=step;
while( ( variable – value2)*step ≤ 0 ) do
{
    <statement 1>
    :
    <statement n>
    variable :=variable+incr;
}
```

- The general form of a **repeat-until** loop:
repeat

<statement 1>

:

<statement n>

until (condition)

- The statements are executed as long as condition is false.

Ex: number:=10 ; sum:=0;

repeat

sum := sum + number;

number := number - 1;

until number = 0;

8. A conditional statement has the following

forms:

if< condition > **then** < statement >

if< condition > **then** < statement 1> **else**

< statement 2>

9. Input and output are done using the instructions **read** and **write**.

10. Procedure or function starts with the word **Algorithm**.

General form :

Algorithm *Name*(<parameter list>)

{

body

}

where *Name* is the name of the procedure.

- Simple variables to functions are passed by value.
- Arrays and records(structure or object) are passed by reference.

Ex-1: Algorithm that finds and returns the maximum of n given numbers.

Algorithm max(a,n)

// a is an array of size n

{

Result:=a[1];

for i:=2 to n do

if a[i]>Result then Result:=a[i];

return Result;

}

Ex-2:-Write an algorithm to sort an array of n integers using bubble sort.

Algorithm sort(a,n)

```
// a is an array of size n
{
    for i:=1 to n-1 do
    {
        for j:=1 to n-1-i do
        {
            if( a[j] > a[j+1] ) then
                t:=a[j]; a[j]:=a[j+1]; a[j+1]:=t;
        }
    }
}
```

Performance Analysis:

- There are many things upon which the performance will depend.
 - Does the program efficiently use primary and Secondary storage?
 - Is the program's running Time acceptable for the task?
 - Does it do what we want it to do?
 - Does it work correctly according to the specifications of the task?
 - Does the program contain documentation that shows how to use it and how it works?
 - Is the program's code readable?

I. Space Complexity:

- The space required by an algorithm is called a space complexity
- The space required by an algorithm has the following components

1. Instruction space. 2. Data space 3.Environmental stack space

1. Instruction space:

- Instruction space is the space needed to store the compiled version of the program instructions.
- The amount of instruction space that is needed depends on the compiler used to compile the program into machine code.

2. Data space:

Data space is the space needed to store all constant and variable values.

Data space has two components.

- i. Space needed by constants

(ex; 1 and 2 in max of n num algorithm) and simple variables(such as i, j, n etc).

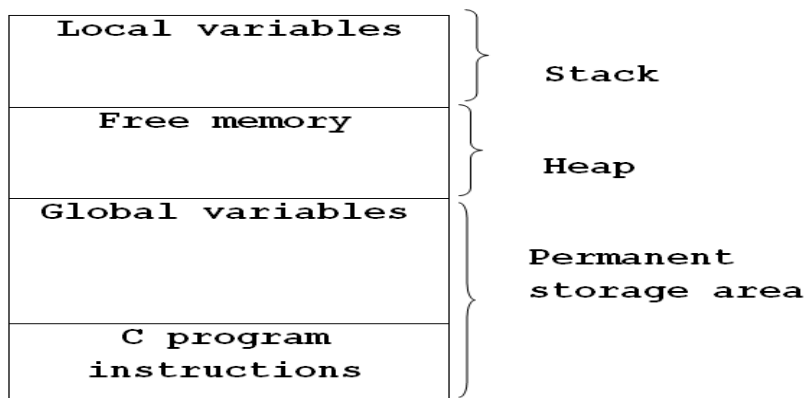
- ii. Space needed by a dynamically allocated objects such as arrays and class instances.
 - Space taken by the variables and constants varies from language to language and platform to platform.

3. Environmental stack space

- The environmental stack is used to save information needed to resume execution of partially completed functions.
- Each time a function is invoked the following data are saved on the environment stack.
 - i. The return address .
 - ii. The values of all local variables and formal parameters in the function being invoked(necessary for recursive functions only).

Memory allocation process

The following figure shows the storage of a program in memory.



Ex: Recursive algorithm

Algorithm rfactorial(n)

```
// n is an integer
{
    fact=1;
    if(n=1 or n=0) return fact;
    else
        fact=n*rfactorial(n-1);
    return fact;
}
```

Each time the recursive function rfactorial is invoked, the current values of n and fact and the program location to return to on completion are saved in the environment stack.

Summary of space complexity:

- The space needed by a program depends on several factors.
- We cannot make an accurate analysis of the space requirements of a program unless we know the computer or compile that will be used.
- However, we can determine the components that depend on the characteristics of the problem instance **(e.x., the number of inputs and outputs or magnitude of the numbers) to be solved.**

Ex:-1 Space requirements of a program that sorts n elements can be expressed as a function of n .

Ex:-2 Space requirements of a program that adds two $m \times n$ matrices can be expressed as a function of m, n .

- The size of the instruction space is independent of the problem instance to be solved.
- The contribution of the constants and simple variables to the data space is also independent of the problem instance to be solved.
- Most of the dynamically allocated memory(ex., arrays, class instances etc) depends on problem instance to be solved.
- The environmental stack space is generally independent of the problem instance unless recursive functions are in use.

Therefore, We can divide the total space needed by Program into two parts:

i) Fixed Space Requirements (C)

Independent of the characteristics of the problem instance (I)

- instruction space
- space for simple variables and constants.

ii) Variable Space Requirements ($S_p(I)$)

depend on the characteristics of the problem instance (I)

Number of inputs and outputs associated with I

recursive stack space (formal parameters, local variables, return address).

Therefore, the space requirement of any problem P can be written as

$$S(p) = C + S_p(\text{Instance characteristics})$$

Note:

- When analyzing the space complexity of an algorithm, we concentrate only on estimating S_p (Instance characteristics).
- We do not concentrate on estimating fixed part C .
- We need to identify the instance characteristics of the problem to measure S_p

Ex-1:

Algorithm sum(a,n)

```
{
    s:=0;
    for i:=1 to n do
        s:=s+a[i];
    return s;
}
```

- Problem instance characterized by n .
- The amount of space needed does not depend on the value of n . Therefore, $S_{\text{sum}}(n)=0$

Ex-2:

Algorithm RSum(a,n)

```
{
    if( $n \leq 0$ ) then return 0;
    else return RSum(a,n-1)+a[n];
}
```

Total no.of recursive calls n , therefore $S_{\text{RSum}}(n)=6(n+1)$

Type	Name	Number of bytes
formal parameter: int	a	2
formal parameter: int	n	2
return address(used internally)		2
Total per one recursive call		6

II. Time Complexity:

$$T(P)=C+T_P(I)$$

- The time, $T(P)$, taken by a program, P , is the sum of its compile time C and its run (or execution) time, $T_P(I)$.
- The compile time does not depend on the instance characteristics.
- We will concentrate on estimating run time $T_P(I)$.
- If we know the characteristics of the compiler to be used, we can determine the
- No. of additions , subtractions, multiplications, divisions, compares, and so on.

Then we can obtain an expression for $T_P(n)$ Of the form

$$T_P(n)=c_a\text{ADD}(n)+c_s\text{SUB}(n)+c_m\text{MUL}(n)+c_d\text{DIV}(n)+\dots\dots\dots$$

Where,

- n denotes the instance characteristics.
- c_a, c_s, c_m, c_d and so on denote the time needed for an addition , subtraction, multiplication, division and so on.
- ADD, SUB, MUL, DIV, and so on are functions whose values are the no.of additions, subtractions, multiplications, divisions, and so on.

- Obtaining such an exact formula is an impossible task, since time needed for an addition, subtraction, and so on, depends on numbers being added, subtracted, and so on.

- The value of **$T_p(n)$** for any given n can be obtained only experimentally.
- Even with the experimental approach, one could face difficulties.
- In a multiuser system the execution time of a program **p** depends on the number of other programs running on the computer at the time program **p** is running.
- As there were some problems in determining the execution time using earlier methods, we will go one step further and count only the number of **program steps**.

Methods to compute the step count:

1) Introduce global variable count into programs with initial value zero.

- Statements to increment count by the appropriate amount are introduced into the program.
- The value of the count by the time program terminates is the number steps taken by the program.

2) Tabular method

- Determine the total number of steps contributed by each statement per execution $\times$ frequency
- Add up the contribution of all statements

Method-I: Introduce variable count into programs

Ex-1:- Iterative sum of n numbers

Algorithm sum(a, n)

```
{
    s:=0;
    count:=count+1; // for assignment statement
    for i:=1 to n do
    {   count:=count+1; // For for
        s:=s+a[i];
        count:=count+1; // for assignment statement
    }
    count:=count+1; // for last time of for
    return s;
    count:=count+1; // for return
}
```

Note: Step count tells us how the run time for a program changes with changes in the instance characteristics.

Ex-2:- Addition of two $m \times n$ matrices

Algorithm Add(a,b,c,,m,n)

```
{
  for i:=1 to m do
  {
    for j:=1 to n do
    {
      c[i,j]:=a[i,j]+b[i,j];
    }
  }
}
```

➔ $2mn + 2m + 1$ steps

Method-II: Tabular method(steps for execution (s/e))

Ex-1:- Iterative sum of n numbers

Statement	s/e	frequency	Total steps
Algorithm sum(a, n)	0	--	0
{	0	--	0
s:=0 ;	1	1	1
for i:=1 to n do	1	n+1	n+1
s:=s+a[i];	1	n	n
return s;	1	1	1
}	0	--	0
Total			2n+3

Ex-2:- Addition of two $m \times n$ matrices

Statement	s/e	frequency	Total steps
Algorithm Add(a,b,c,m, n)	0	--	0
{	0	--	0
for i:=1 to m do	1	m+1	m+1
for j:=1 to n do	1	m(n+1)	mn+m
c[i,j]:=a[i,j]+b[i,j] ;	1	mn	mn
}	0	--	0
Total			2mn+2m+1

Best, Worst, Average Cases

- ✓ Not all inputs of a given size take the same number of program steps.
- ✓ Sequential search for K in an array of n integers:
 - Begin at first element in array and look at each element in turn until K is found.

1. **Best-Case Step count:-** Minimum number of steps executed by the algorithm for the given parameters.
2. **Worst-Case Step count:-** Maximum number of steps executed by the algorithm for the given parameters.
3. **Average-Case Step count:-** Average number of steps executed by an algorithm.

Asymptotic Notations:

Asymptotic notation describes the behavior of functions for the large inputs.

1. Big Oh(O) notation: The big oh notation describes an upper bound on the asymptotic growth rate of the function f .

Definition: [Big “oh”]

- $f(n) = O(g(n))$
- (read as “ f of n is big oh of g of n ”) iff there exist positive constants ‘ c ’ and n_0 such that

$$f(n) \leq cg(n) \text{ for all } n, n \geq n_0.$$

- The definition states that the function $f(n)$ is **at most** c times the function $g(n)$ except when n is smaller than n_0 .
- In other words, $f(n)$ grows slower than or same rate as $g(n)$.
- When providing an upper –bound function g for f , we normally use a single term in n .

Examples:

- $f(n) = 3n+2$
 - $3n + 2 \leq 4n$, for all $n \geq 2$, $\therefore 3n + 2 = O(n)$
- $f(n) = 10n^2+4n+2$
 - $10n^2+4n+2 \leq 11n^2$, for all $n \geq 5$, $\therefore 10n^2+4n+2 = O(n^2)$
- $f(n)=6*2^n+n^2=O(2^n)$ /* $6*2^n+n^2 \leq 7*2^n$ for $n \geq 4$ */

- It also possible to write $10n^2+4n+2 = O(n^3)$ since $10n^2+4n+2 \leq 7n^3$ for $n \geq 2$
- Although n^3 is an upper bound for $10n^2+4n+2$, it is not a tight upper bound; we can find a smaller function (n^2) that satisfies big oh relation.

2. Omega (Ω) notation:

- The omega notation describes a lower bound on the asymptotic growth rate of the function f .

Definition: [Omega]

- $f(n) = \Omega(g(n))$
- (read as “ f of n is omega of g of n ”)
- iff there exist positive constants c and n_0 such that
- $f(n) \geq cg(n)$ for all n , $n \geq n_0$.

- The definition states that the function $f(n)$ is **at least** c times the function $g(n)$ except when n is smaller than n_0 .
- In other words, $f(n)$ grows faster than or same rate as $g(n)$.

Examples

$$f(n) = 3n+2 \quad 3n+2 \geq 3n, \text{ for all } n \geq 1, \therefore 3n+2 = \Omega(n)$$

$$f(n) = 10n^2+4n+2 \quad 10n^2+4n+2 \geq n^2, \text{ for all } n \geq 1, \therefore 10n^2+4n+2 = \Omega(n^2)$$

- It is also possible to write $10n^2+4n+2 = \Omega(n)$ since $10n^2+4n+2 \geq n$ for $n \geq 0$
- Although n is a lower bound for $10n^2+4n+2$, it is not a tight lower bound; we can find a larger function (n^2) that satisfies omega relation.
- But, we can not write $10n^2+4n+2 = \Omega(n^3)$, since it does not satisfy the omega relation for sufficiently large input.

3. Theta (Θ) notation:

- The Theta notation describes a **tight bound** on the **asymptotic growth rate** of the function f .

– **Definition:** [Theta]

- $f(n) = \Theta(g(n))$
- (read as “ f of n is theta of g of n ”)
- iff there exist positive constants c_1 , c_2 , and n_0 such that
 - $c_1g(n) \leq f(n) \leq c_2g(n)$ for all n , $n \geq n_0$.

- The definition states that **the function $f(n)$ lies between c_1 times the function $g(n)$ and c_2 times the function $g(n)$** except when n is smaller than n_0 .
- In other words, **$f(n)$ grows same rate as $g(n)$.**

Examples:-

$$f(n) = 3n+2 \quad 3n \leq 3n+2 \leq 4n, \text{ for all } n \geq 2, \therefore 3n+2 = \Theta(n)$$

$$f(n) = 10n^2+4n+2 \quad n^2 \leq 10n^2+4n+2 \leq 11n^2, \text{ for all } n \geq 5, \therefore 10n^2+4n+2 = \Theta(n^2)$$

- But, we can not write either $10n^2+4n+2 = \Theta(n)$ or $10n^2+4n+2 = \Theta(n^3)$, since neither of these will satisfy the theta relation.

4. Little Oh(O) notation:

- Little o notation is used to describe an upper bound that cannot be tight. In other words, loose upper bound of $f(n)$.
- Let $f(n)$ and $g(n)$ are the functions that map positive real numbers. We can say that the function $f(n)$ is $o(g(n))$ if for any real positive constant c , there exists an integer constant $n_0 \leq 1$ such that $f(n) > 0$.

Definition: [Little “oh”]

– $f(n) = o(g(n))$ (read as “ f of n is little oh of g of n ”) iff $f(n) < c g(n)$ for all non negative values of ‘ n ’, where $n \geq n_0$.

- The definition states that the function $f(n)$ is less than c times the function $g(n)$ except when n is smaller than n_0 .
- In other words, $f(n)$ grows slower than $g(n)$.

Examples: $f(n) = 3n+2 = o(n^2)$ Since *However*, $3n+2 \neq o(n)$

Big-Oh, Theta, Omega and Little-oh:

Tips :

- Think of **$O(g(n))$** as “less than or equal to” $g(n)$
 - **Upper bound:** “grows slower than or same rate as” $g(n)$
- Think of **$\Omega(g(n))$** as “greater than or equal to” $g(n)$
 - **Lower bound:** “grows faster than or same rate as” $g(n)$
- Think of **$\Theta(g(n))$** as “equal to” $g(n)$
 - **“Tight” bound: same growth rate**
- Think of **$o(g(n))$** as “less than to” $g(n)$
 - Strict Upper bound: “grows slower than ” $g(n)$
 - (True for large N)

Functions ordered by growth rate:

<u>Function</u>	<u>Name</u>
1	Growth is constant
logn	Growth is logarithmic
n	Growth is linear
nlogn	Growth is n-log-n
n²	Growth is quadratic
n³	Growth is cubic
2ⁿ	Growth is exponential
n!	Growth is factorial

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

To get a feel for how the various functions grow with n , you are advised to study the following figs:

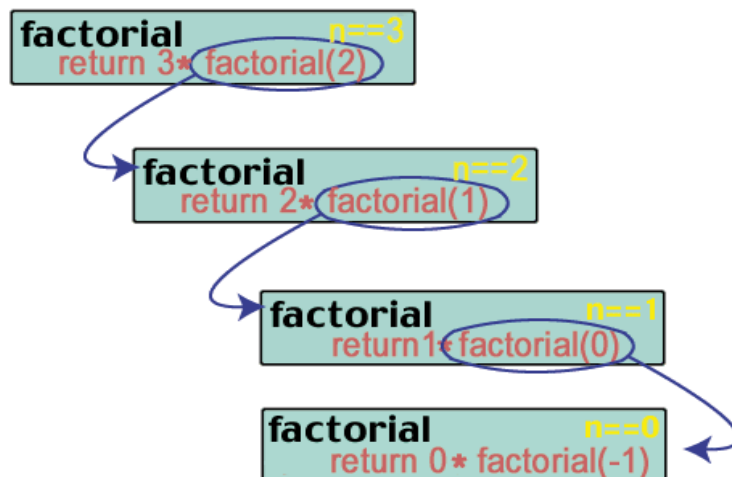
Recursion:

- Recursion is a powerful algorithmic technique in which a function calls itself (either directly or indirectly) on a smaller problem of the same type in order to simplify the problem to a solvable state.
- Every recursive function must have at least two cases: the recursive case and the base case.

- The base case is a small problem that we know how to solve and is the case that causes the recursion to end.
- The recursive case is the more general case of the problem we're trying to solve.

As an example, with the factorial function $n!$, the recursive case is $n! = n * (n - 1)!$ and the base case is $n = 1$ when $n = 0$ or $n = 1$.

- Recursive techniques can often present simple and elegant solutions to problems.
- However, they are not always the most efficient. Recursive functions often use a good deal of memory and stack space during their operation.
- The stack space is the memory set aside for a program to use to keep track of all of the functions and their local states currently in the middle of execution.
- Because they are easy to implement but relatively inefficient, recursive solutions are often best used in cases where development time is a significant concern.
- Sometimes a problem is too difficult or too complex to solve because it is too big.
- If the problem can be broken down into smaller versions of itself, we may be able to find a way to solve one of these smaller versions and then be able to build up to a solution to the entire problem.
- This is the idea behind recursion; recursive algorithms break down a problem into smaller pieces which you either already know the answer to, or can solve by applying the same algorithm to each piece, and then combining the results.
- Stated more concisely, a recursive definition is defined in terms of itself.



Types of Recursions:

Recursion is a process in which a method calls itself to solve a problem. In Java, recursion can be broadly classified into two main types:

1. Direct Recursion:

- **Description:** When a method calls itself directly, it is known as direct recursion. This is the most common form of recursion.

Example: Factorial of a number.

```
public class DirectRecursionExample
{
    // Method to calculate factorial of a number
    public static int factorial(int n)
    {
        if (n == 0) { // Base condition to stop recursion
            return 1;
        }
        return n * factorial(n - 1); // Recursive call
    }
    public static void main(String[] args)
    {
        int number = 5;
        int result = factorial(number);
        System.out.println("Factorial of " + number + " is: " + result);
    }
}
```

2. Indirect Recursion:

- **Description:** In indirect recursion, a method calls another method, which eventually leads back to the first method.

Example: Two methods calling each other.

```
public class IndirectRecursionExample
{
    // Method A
    public static void methodA(int n)
    {
        if (n > 0)
        {
            System.out.println("In method A: " + n);
            methodB(n - 1); // Calls methodB
        }
    }
    // Method B
    public static void methodB(int n) {
        if (n > 0) {
            System.out.println("In method B: " + n);
        }
    }
}
```

```
        methodA(n - 1); // Calls methodA, creating indirect recursion
    }
}
public static void main(String[] args) {
    methodA(5); // Start recursion
}
}
```

3. Tail Recursion

- **Description:** Tail recursion is a special case of recursion where the recursive call is the last statement in the method. It is optimized by the compiler or runtime, leading to better performance.

Example: Tail-recursive factorial.

```
public class TailRecursionExample
{
    // Tail-recursive method to calculate factorial
    public static int tailFactorial(int n, int result)
    {
        if (n == 0)
        {
            return result;
        }
        return tailFactorial(n - 1, n * result); // Tail-recursive call
    }

    public static void main(String[] args)
    {
        int number = 5;
        int result = tailFactorial(number, 1); // Start with result as 1
        System.out.println("Factorial of " + number + " is: " + result);
    }
}
```

4. Non-Tail Recursion:

- **Description:** In non-tail recursion, the recursive call is not the last operation performed. The method needs to perform additional operations after the recursive call.

Example: Fibonacci series.

```
public class NonTailRecursionExample
{
```

```
// Method to calculate nth Fibonacci number
public static int fibonacci(int n)
{
    if (n <= 1)
    {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2); // Non-tail recursive call
}
public static void main(String[] args) {
    int number = 5;
    int result = fibonacci(number);
    System.out.println("Fibonacci of " + number + " is: " + result);
}
}
```

5. Mutual Recursion:

- **Description:** This is a form of indirect recursion where two or more methods call each other in a mutually recursive way.

Example: Odd and even number checker using mutual recursion.

```
public class MutualRecursionExample
{
    // Method to check if a number is even
    public static boolean isEven(int n)
    {
        if (n == 0)
        {
            return true; // Base case
        }
        return isOdd(n - 1); // Mutual recursion
    }
    // Method to check if a number is odd
    public static boolean isOdd(int n)
    {
        if (n == 0)
        {
            return false; // Base case
        }
        return isEven(n - 1); // Mutual recursion
    }
}
public static void main(String[] args)
{
    int number = 5;
    if (isEven(number))
    {

```

```
        System.out.println(number + " is even.");
    } else {
        System.out.println(number + " is odd.");
    }
}
```

Applications:

1. Fibonacci Series
2. Climbing Stairs
3. Reverse String
4. Happy Number
5. GCD(Greatest common Divisor)
6. Strobogrammatic number II:

1. Fibonacci Series:

- A **Fibonacci Series** in Java is a series of numbers in which the next number is the sum of the previous two numbers.
- The first two numbers of the Fibonacci series are 0 and 1. The Fibonacci numbers are significantly used in the computational run-time study of an algorithm to determine the greatest common divisor of two integers.
- In arithmetic, the Wythoff array is an infinite matrix of numbers resulting from the Fibonacci sequence.

The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Time Complexity:

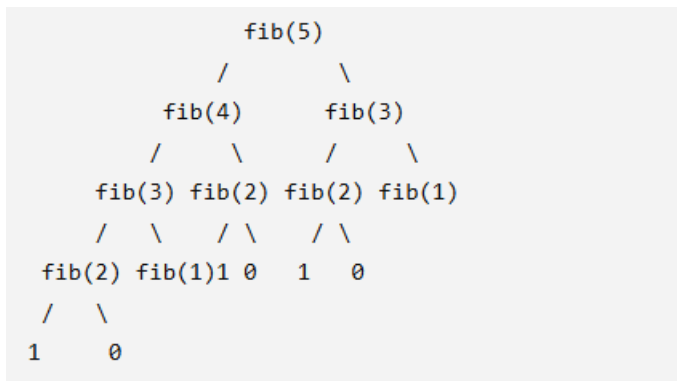
For the **Fibonacci Series using General Recursion**, the time complexity is **$O(2^n)$** .

Each call makes **two recursive calls**:

- fibonacci(n - 1)
- fibonacci(n - 2)

This creates a recursion tree where many subproblems are computed repeatedly.

Recursion Tree for n = 5



Notice that:

- `fib(3)` is computed **twice**.
- `fib(2)` is computed **three times**.
- `fib(1)` and `fib(0)` are computed multiple times.

These repeated computations cause the exponential running time.

Complexity Analysis

Complexity	Value
Time Complexity	$O(2^n)$
Space Complexity	$O(n)$ (maximum recursion depth)

Explanation

- At each level, the function branches into **two recursive calls**.
- The recursion tree grows exponentially.
- Hence, the total number of function calls is approximately 2^n .

Java Program for Fibonacci Series Using General Recursion: FibonacciCalc.java

```

public class FibonacciCalc
{
    public static int fibonacciRecursion(int n)
    {
        if(n == 0)
        {
            return 0;
        }
        if(n == 1 || n == 2)
        {
            return 1;
        }
    }
}

```

```

        }
        return fibonacciRecursion(n-2) + fibonacciRecursion(n-1);
    }
    public static void main(String args[])
    {
        int maxNumber = 10;
        System.out.print("Fibonacci Series of "+maxNumber+" numbers: ");
        for(int i = 0; i < maxNumber; i++)
        {
            System.out.print(fibonacciRecursion(i) + " ");
        }
    }
}

```

Output: Fibonacci Series of 10 numbers: 0 1 1 2 3 5 8 13 21 34

2. Climbing Stairs:

- Given a staircase of **N** steps and you can either climb 1 or 2 steps at a given time. The task is to return the count of distinct ways to climb to the top. **Note:** The order of the steps taken matters.

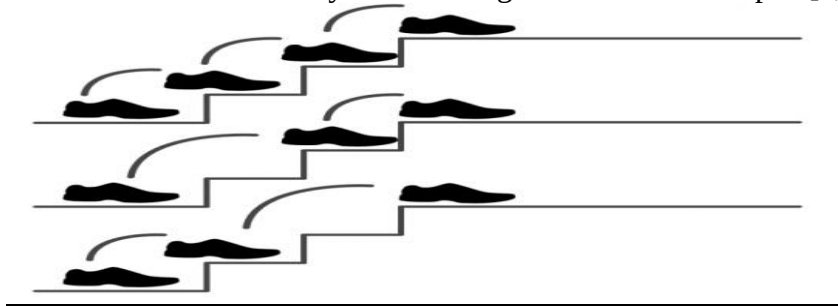
Examples:

Input: N=3

Output: 3

Explanation:

There are three distinct ways of climbing a staircase of 3 steps : [1, 1, 1], [2, 1] and [1, 2].



Input: N=2

Output: 2

Explanation:

There are two distinct ways of climbing a staircase of 3 steps: [1, 1] and [2].

Input: n = 4

Output: 5

(1, 1, 1, 1), (1, 1, 2), (2, 1, 1), (1, 2, 1), (2, 2)

Method: Brute Force (Recursive) Approach

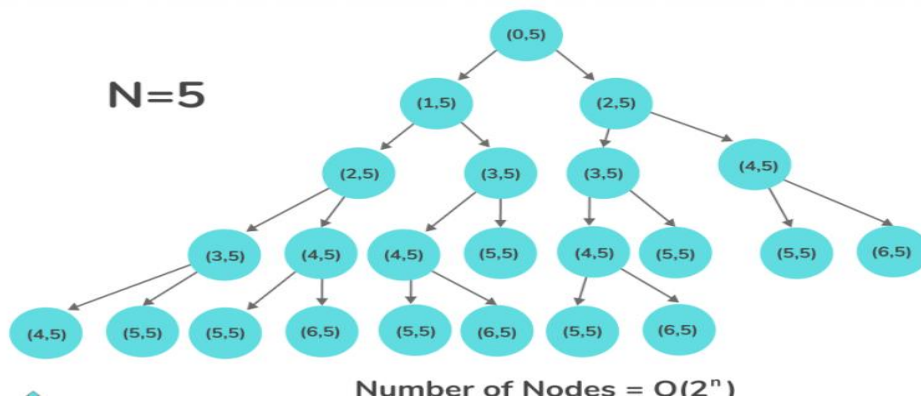
The approach is to consider all possible combination steps i.e. 1 and 2, at every step. To reach the **Nth** stair, one can jump from either **(N – 1)th** or from **(N – 2)th** stair. Hence, for each step, total ways would be the summation of **(N – 1)th stair + (N – 2)th stair**.

The recursive function would be:

$$\text{ClimbStairs}(N) = \text{ClimbStairs}(N - 1) + \text{ClimbStairs}(N - 2).$$

If we observe carefully, the expression is nothing but the **Fibonacci Sequence**.

Sample Recursive Tree for **N = 5**:

**Algorithm:**

- If $N < 2$, return 1. This is the termination condition for the function.
- Else, find the summation of **ClimbStairs(N – 1) + ClimbStairs(N – 2)**.

Complexity	Value
Time Complexity	$O(2^n)$
Space Complexity	$O(n)$ (maximum recursion depth)

Java program for climbing Stairs using General Recursion: Stairs_M1.java

```
import java.util.*;

class Stairs_M1 {
    // A simple recursive program to find
    // n'th fibonacci number

    static int ClimbStairs(int n)
    {
        if (n <= 2)
            return n;
    }
}
```

```
        return ClimbStairs(n-1) + ClimbStairs(n-2);
    }

    // Returns number of ways to reach s'th stair
    static int countWays(int s)
    {
        return ClimbStairs(s);
    }

    public static void main(String args[])
    {
        Scanner sc =new Scanner(System.in);
        System.out.println("enter no.of stairs");
        int s = sc.nextInt();
        System.out.println("Number of ways = " + countWays(s));
    }
}
```

3. Reverse String:

Given a string *s*, reverse the order of its characters and return the resulting string.

Your solution **must** use **general recursion**, where each recursive call processes a smaller portion of the string and combines the results while returning from the recursion. Iterative approaches and built-in string reversal methods are not allowed.

Example 1:

Input : *s* = "hello"

Output: "olleh"

Explanation: The characters of the string are reversed recursively to produce "olleh".

Example 2

Input: *s* = "OpenAI"

Output: "IAnepO"

Example 3

Input: *s* = "a"

Output: "a"

Explanation: A string containing a single character remains unchanged.

Approach

1. Read the input string from the user using `Scanner`.
2. Define a recursive function `reverseString(String s)` to reverse the string.
3. **Base Case:** If the string has **0 or 1 character**, return the string as it is since it is already reversed.
4. **Recursive Case:**
 - Recursively reverse the substring starting from the second character (`s.substring(1)`).
 - Append the first character (`s.charAt(0)`) to the end of the reversed substring.
5. Return the final reversed string and print it.

Algorithm

1. Read the input string `s`.
2. If `s.length() <= 1`, return `s`.
3. Otherwise, return: `reverseString(s.substring(1)) + s.charAt(0)`;
4. Print the returned string.

Dry Run for Input "ABC"

```
reverseString("ABC")
= reverseString("BC") + 'A'
= (reverseString("C") + 'B') + 'A'
= ("C" + 'B') + 'A'
= "CBA"
```

Time Complexity Analysis:

The recursion itself makes ***n*** calls, but the use of `substring()` and string concatenation (+) makes the overall complexity ***O(n²)***. Suppose the string is "ABCDE" (length = 5). The recursive calls are:

```
reverse("ABCDE")
reverse("BCDE")
reverse("CDE")
reverse("DE")
reverse("E")
```

Cost of substring(): If `substring()` creates a new string (as in modern Java versions), then:

- `substring(1)` from "ABCDE" copies **4** characters.
- `substring(1)` from "BCDE" copies **3** characters.
- `substring(1)` from "CDE" copies **2** characters.
- `substring(1)` from "DE" copies **1** character.

Total work: $4 + 3 + 2 + 1 = 10$

For a string of length ***n***: $(n-1) + (n-2) + \dots + 2 + 1$

$$= n(n-1)/2$$

$$= O(n^2)$$

also creates a **new String** at every return. Each concatenation copies the characters accumulated so far.

For "ABCDE":

```
"E"           -> copy 1 character
"ED"          -> copy 2 characters
"EDC"         -> copy 3 characters
"EDCB"        -> copy 4 characters
"EDCBA"       -> copy 5 characters
```

Total work: $1 + 2 + 3 + 4 + 5 = 15 = O(n^2)$

Therefore

- Recursive calls: **$O(n)$**
- `substring()` copying: **$O(n^2)$**
- String concatenation: **$O(n^2)$**

Overall complexity: Time: $O(n^2)$ and Space: $O(n)$ (recursion stack, ignoring the extra temporary strings)

Java program for string reverse using recursion:**ReverseString.java**

```
import java.util.Scanner;

public class ReverseString {

    // General recursive function to reverse a string
    public static String reverseString(String s) {
        // Base case
        if (s.length() <= 1) {
            return s;
        }
        // Recursive case
        return reverseString(s.substring(1)) + s.charAt(0);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read input from the command line
        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        // Reverse the string
        String reversed = reverseString(str);

        // Display the result
        System.out.println("Reversed String: " + reversed);

        sc.close();
    }
}
```

Sample Input: Enter a string: Hello World

Sample Output: Reversed String: dlroW olleH

4. Happy number:

- A happy number is a number which eventually reaches 1 when replaced by the sum of the square of each digit.
- Whereas if during this process any number gets repeated, the cycle will run infinitely and such numbers are called unhappy numbers.

Example-1: 13 is a happy number because,

$$1^2 + 3^2 = 10 \text{ and,}$$

$$1^2 + 0^2 = 1$$

On the other hand, 36 is an unhappy number.

Example-2:

$$28 = 2^2 + 8^2 = 4 + 64 = 68$$

$$68 = 6^2 + 8^2 = 36 + 64 = 100$$

$$100 = 1^2 + 0^2 + 0^2 = 1 + 0 + 0 = 1$$

Hence, 28 is a happy number.

Example-3 :

$$12 = 1^2 + 2^2 = 1 + 4 = 5$$

Hence, 12 is not a happy number.

Time Complexity: $O(\log n)$

Reason:

1. The `sumOfDigits()` method processes each digit of the number once.
2. A number with d digits has $d = \log_{10}(n)$ digits.
3. Therefore, one call to `sumOfDigits()` takes **$O(\log n)$** time.
4. The recursive function `isHappy()` makes only **one recursive call** at a time (not two like Fibonacci).
5. The number quickly reduces to a small value (for example, $19 \rightarrow 82 \rightarrow 68 \rightarrow 100 \rightarrow 1$), so the number of recursive calls is bounded by a small constant for fixed-size integers.

Hence, the overall complexity is:

- **Time Complexity: $O(\log n)$**
- **Space Complexity: $O(\log n)$** (due to the recursion call stack)

Java program for Happy number using recursion: Happynumber.java

```
import java.util.Scanner;
```

```
class Happynumber
```

```
{  
    private static boolean isHappy(int num)  
    {
```

```
    if (num == 1)
        return true;
    if (num == 4)
        return false;
    return isHappy(sumOfDigits(num));
}
private static int sumOfDigits(int num)
{
    int sum = 0;
    while (num > 0)
    {
        int digit = num % 10;
        sum += digit * digit;
        num /= 10;
    }
    return sum;
}
public static void main(String[] args)
{
    Scanner in = new Scanner(System.in);
    System.out.print("Enter a number: ");
    int num = in.nextInt();
    if (isHappy(num))
        System.out.print("Happy Number");
    else
        System.out.print("Not a Happy Number");
    in.close();
}
}
```

Example 1:**Input=** Enter a number: 13**Output=** Happy Number**Example 2:****Input=** Enter a number:36**Output=** Not a Happy number**Complete Trace for Input 19 Using Your Java Code Logic****isHappy(19)****1st Call:** sumOfDigits(19)

num = 19

sum = 0

Iteration 1

digit = 19 % 10 = 9

$\text{sum} = 0 + (9 \times 9) = 81$
 $\text{num} = 19 / 10 = 1$

Iteration 2

 $\text{digit} = 1 \% 10 = 1$
 $\text{sum} = 81 + (1 \times 1) = 82$
 $\text{num} = 1 / 10 = 0$
Loop Ends: Return 82

Recursive call:

isHappy(19)
↓
isHappy(82)

2nd Call: `sumOfDigits(82)`

 $\text{num} = 82$
 $\text{sum} = 0$

Iteration 1

 $\text{digit} = 82 \% 10 = 2$
 $\text{sum} = 0 + (2 \times 2) = 4$
 $\text{num} = 82 / 10 = 8$

Iteration 2

 $\text{digit} = 8 \% 10 = 8$
 $\text{sum} = 4 + (8 \times 8) = 68$
 $\text{num} = 8 / 10 = 0$
Loop Ends: Return 68

Recursive call:

isHappy(82)
↓
isHappy(68)

3rd Call: `sumOfDigits(68)`

 $\text{num} = 68$
 $\text{sum} = 0$

Iteration 1

 $\text{digit} = 68 \% 10 = 8$
 $\text{sum} = 0 + (8 \times 8) = 64$
 $\text{num} = 68 / 10 = 6$

Iteration 2

digit = $6 \% 10 = 6$
sum = $64 + (6 \times 6) = 100$
num = $6 / 10 = 0$
Loop Ends: Return 100

Recursive call:

isHappy(68)



isHappy(100)

4th Call: sumOfDigits(100)

num = 100
sum = 0

Iteration 1

digit = $100 \% 10 = 0$
sum = $0 + (0 \times 0) = 0$
num = $100 / 10 = 10$

Iteration 2

digit = $10 \% 10 = 0$
sum = $0 + (0 \times 0) = 0$
num = $10 / 10 = 1$

Iteration 3

digit = $1 \% 10 = 1$
sum = $0 + (1 \times 1) = 1$
num = $1 / 10 = 0$

Loop Ends: Return 1

Recursive call:

isHappy(100)



isHappy(1)

5th Call: Java code:

isHappy(1)

if(num == 1)

return true;

5. GCD (Greatest Common Divisor)

Given two positive integers a and b , return their **Greatest Common Divisor (GCD)**.

The **Greatest Common Divisor (GCD)** of two integers is the largest positive integer that divides both numbers without leaving a remainder.

Your solution **must** use **recursion** to compute the GCD. Iterative solutions and built-in GCD functions are not allowed.

Example 1

Input:

$a = 48$

$b = 18$

Output:

6

Explanation:

The common divisors of 48 and 18 are 1, 2, 3, 6. The greatest among them is 6.

Example 2

Input:

$a = 24$

$b = 36$

Output:

12

Example 3

Input:

$a = 17$

$b = 13$

Output:

1

Explanation:

Since 17 and 13 have no common divisor other than 1, their GCD is 1.

General Procedure: For example – Let's take two numbers **63** and **21**.

- Factors of 63 – **3, 7, 9, 21 and 63**
- Factors of 21 – **3, 7, 21**
- The common divisors of both the numbers are **3, 7, 21**. Out of which the greatest common divisor is **21**.
- So the GCD (63,21) is **21**.

Euclidean Algorithm:

The **Euclidean Algorithm** is an efficient method for finding the **Greatest Common Divisor (GCD)** of two integers.

It is based on the principle: **The GCD of two numbers does not change if the larger number is replaced by its remainder when divided by the smaller number.**

Mathematically, $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$, where $b \neq 0$

The recursion continues until the second number becomes **0**. $\text{GCD}(a, 0) = a$

Pseudo Code of the Algorithm-

Step 1: **Let a, b be the two numbers**

Step 2: **a mod b = R**

Step 3: **Let a = b and b = R**

Step 4: **Repeat Steps 2 and 3 until a mod b is greater than 0**

Step 5: **GCD = b**

Step 6: Finish

Example: Find the GCD of **48** and **18**.

$\text{GCD}(48, 18)$
 $= \text{GCD}(18, 48 \% 18)$
 $= \text{GCD}(18, 12)$

 $= \text{GCD}(12, 18 \% 12)$
 $= \text{GCD}(12, 6)$

 $= \text{GCD}(6, 12 \% 6)$
 $= \text{GCD}(6, 0)$
 $= 6$

Hence, $\text{GCD}(48, 18) = 6$

Time Complexity Explanation:

In each recursive call:

- The larger number is replaced by its remainder after division.
- The remainder is always **smaller** than the divisor.
- Therefore, the values decrease rapidly.

The Euclidean algorithm requires at most **$O(\log(\min(a, b)))$** recursive calls.

Complexity Analysis

- **Time Complexity:** **$O(\log(\min(a, b)))$**
- **Space Complexity:** **$O(\log(\min(a, b)))$** (recursive call stack)

java program for Find GCD of Two Numbers using Recursion: GCD.java

```
import java.util.*;

public class GCD
{
    public static int calculateGCD(int a, int b){
        //If both the number are equal
        if (a == b) {
            return a;
        }

        /* If a is equal to zero then return b */
        else if (a == 0) {
            return b;
        }
        /* If b is equal to zero then return a */
        else if (b == 0) {
            return a;
        }
        else if (a > b) {
            //Recursive call
            return calculateGCD(a % b, b);
        } else {
            //Recursive call
            return calculateGCD(a, b % a);
        }
    }

    public static void main(String[] args)
    {
        Scanner in = new Scanner(System.in);
        int a= in.nextInt();
        int b= in.nextInt();
        System.out.println("GCD of " + a + " and " + b + " is: " + calculateGCD(a, b));
    }
}
```

Example:

input= 63 21

output= GCD of 63 and 21 is: 21

input= 23 0

output= GCD of 23 and 0 is: 23

input= 0 24

output= GCD of 0 and 24 is: 24

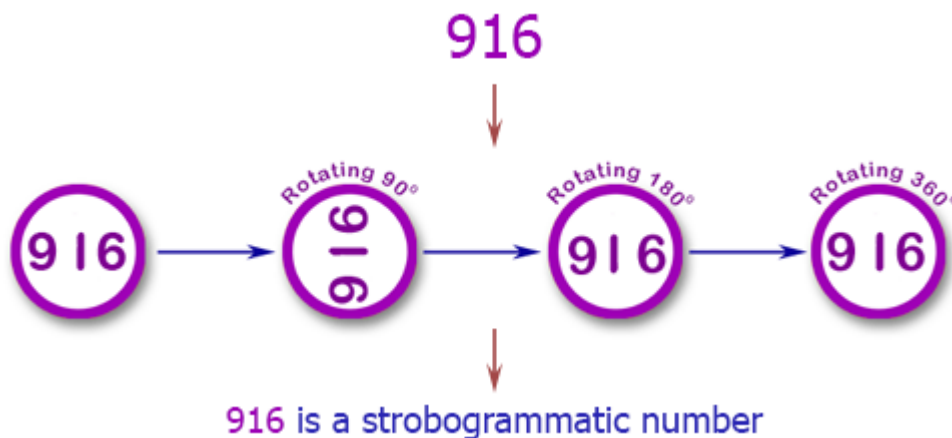
6. Strobogrammatic number II:

- A strobogrammatic number is a number whose numeral is rotationally symmetric, so that it appears the same when rotated 180 degrees.
- In other words, the numeral looks the same right-side up and upside down (e.g., 69, 96, 1001).
- A strobogrammatic prime is a strobogrammatic number that is also a prime number, i.e., a number that is only divisible by one and itself (e.g., 11).
- It is a type of ambigram, words and numbers that retain their meaning when viewed from a different perspective, such as palindromes."

The first few strobogrammatic numbers are :

0, 1, 8, 11, 69, 88, 96, 101, 111, 181, 609, 619, 689, 808, 818, 888, 906, 916, 986, 1001, 1111, 1691, 1881, 1961, 6009, 6119, 6699, 6889, 6969, 8008, 8118, 8698, 8888, 8968, 9006, 9116, 9696, 9886, 9966, ...

Pictorial Presentation:



Let us first enumerate the cases where n is 0,1,2,3,4:

```
n = 0: none
n = 1: 0, 1, 8
n = 2: 11, 69, 88, 96
n = 3: 101, 609, 808, 906, 111, 619, 818, 916, 181, 689, 888, 986
n = 4: 1001, 6009, 8008, 9006, 1111, 6119, 8118, 9116, 1691, 6699, 8698, 9696, 1881, 6889,
8888, 9886, 1961, 6969, 8968, 9966
```

Observations:

- Look at $n=0$ and $n=2$, you can find that the latter is based on the former, and the left and right sides of each number are increased by [1 1], [6 9], [8 8], [9 6]
- Look at $n=1$ and $n=3$, it's more obvious, increase [1 1] around 0, become 101, increase around 0 [6 9], become 609, increase around 0 [8 8], Becomes 808, increases [9 6] to the left and right of 0, becomes 906, and then adds the four sets of numbers to the left and right sides of 1 and 8 respectively
- In fact, it starts from the $m=0$ layer and adds layer by layer. It should be noted that when the n layer is added.
- [0 0] cannot be added to the left and right sides, because 0 cannot appear at the beginning of two or more digits. In the process of recursive in the middle, it is necessary to add 0 to the left and right sides of the number.

Approach

The idea is to **construct the strobogrammatic numbers recursively from the center outward.**

1. Base Cases

- If $n == 0$, return a list containing an empty string ("").
- If $n == 1$, return the single-digit strobogrammatic numbers: ["0", "1", "8"].

2. Recursive Step

- Recursively generate all strobogrammatic numbers of length $n - 2$.
- For each generated string, add the following valid digit pairs around it:
 - "0" + s + "0" (only if it is **not** the outermost level to avoid leading zeros)
 - "1" + s + "1"
 - "6" + s + "9"
 - "8" + s + "8"
 - "9" + s + "6"

3. Continue the recursion until the required length n is obtained.
4. Return the list of all generated strobogrammatic numbers.

Algorithm

1. If $n == 0$, return [""].
2. If $n == 1$, return ["0", "1", "8"].
3. Recursively generate all strings of length $n - 2$.
4. Surround each generated string with the valid strobogrammatic digit pairs.
5. Skip "0...0" when constructing the outermost layer.
6. Return the generated list.

Time Complexity

At each recursive level, **up to 5 valid digit pairs** can be added around every generated string.

- Number of recursive levels = $n/2$
- Branching factor = 5

Therefore,

- **Time Complexity: $O(5^{n/2})$**

Java program for Strobogrammatic Number using recursion: StrobogrammaticNumber .java

```
import java.util.*;

public class StrobogrammaticNumber {

    // Recursive function
    public static List<String> findStrobogrammatic(int n) {
        return helper(n, n);
    }

    private static List<String> helper(int n, int totalLength) {

        // Base case
        if (n == 0) {
            return Arrays.asList("");
        }

        // Base case
        if (n == 1) {
            return Arrays.asList("0", "1", "8");
        }

        // Recursive call
        List<String> list = helper(n - 2, totalLength);
        List<String> result = new ArrayList<>();

        for (String s : list) {

            // Do not place 0 at the beginning
            if (n != totalLength) {
                result.add("0" + s + "0");
            }

            result.add("1" + s + "1");
            result.add("6" + s + "9");
            result.add("8" + s + "8");
            result.add("9" + s + "6");
        }

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter the length (n): ");  
int n = sc.nextInt();
```

```
List<String> numbers = findStrobogrammatic(n);
```

```
// Print in array format  
System.out.println(numbers);
```

```
sc.close();
```

```
}
```

```
}
```

Sample Input and Output:

Example-1:

Input = Enter the length (n): 1

output=

[0, 1, 8]

Example =2

Input = Enter the length (n): 2

output=

[11, 69, 88, 96]

Example =3

Input = Enter the length (n): 3

output=

[101, 609, 808, 906, 111, 619, 818, 916, 181, 689, 888, 986]

Example =4

Input = Enter the length (n): 4

output=

[1001, 6009, 8008, 9006, 1111, 6119, 8118, 9116, 1691, 6699, 8698, 9696, 1881, 6889, 8888, 9886, 1961, 6969, 8968, 9966]

Example =5

input= Enter the length (n): 5

output=

[10001, 60009, 80008, 90006, 11011, 61019, 81018, 91016, 16091, 66099, 86098, 96096, 18081, 68089, 88088, 98086, 19061, 69069, 89068, 99066, 10101, 60109, 80108, 90106, 11111, 61119, 81118, 91116, 16191, 66199, 86198, 96196, 18181, 68189, 88188, 98186, 19161, 69169, 89168, 99166, 10801, 60809, 80808, 90806, 11811, 61819, 81818, 91816, 16891, 66899, 86898, 96896, 18881, 68889, 88888, 98886, 19861, 69869, 89868, 99866]

Divide and conquer:**General method:**

- D&C Technique splits n inputs into k subsets, $1 < k \leq n$, generating k sub problems.
- These sub problems will be solved and then combined by using a separate method to get solution to the whole problem.
- If the sub problems are large, then the D&C Technique will be reapplied.
- Often sub problems getting from the D&C Technique are of the same type as the original problem.
- The reapplication of the D&C Technique is naturally expressed by a recursive algorithm.
- Now smaller and smaller problems of the same kind are generated until subproblems that are small enough to solve without splitting further.

Control Abstraction / General Method for Divide and Conquer Technique:**Algorithm DAndC(p)**

```

{
    if Small(p) then return s(p);
    else
    {
        Divide p into smaller problems p1,p2,.....,pk, k≥1;
        Apply D&C to each of these subproblems;
        return Combine(DAndC(p1), DAndC(p2),.....,DAndC(pk));
    }
}

```

- If the size of p is n and the sizes of the k subproblems are $n_1, n_2, \dots, n_k$, then the computing time of D&C is described by the recurrence relation

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{Otherwise} \end{cases}$$

- Where $T(n)$ is the time for D&C on any input of size n and $g(n)$ is the time to compute the answer directly for small inputs.
- The function $f(n)$ is the time for dividing p and combining the solutions of subproblems.
- The Time Complexity of many D&C algorithms is given by recurrences of the form

$$T(n) = \begin{cases} n & \text{small} \\ aT(n/b) + f(n) & \text{Otherwise} \end{cases}$$

- Where a , b and c are known constants, and n is a power of b (i.e $n = b^k$)

Master Theorem for Divide and Conquer Recurrences

- All divide and conquer algorithms (also discussed in detail in the Divide and Conquer chapter) divide the problem into sub-problems, each of which is part of the original problem, and then perform some additional work to compute the final answer.
- As an example, a merge sort algorithm [for details, refer to Sorting chapter] operates on two sub-problems, each of which is half the size of the original, and then performs $O(n)$ additional work for merging.
- This gives the running time equation:

$$T(n) = 2T(n/2) + O(n)$$
- The following theorem can be used to determine the running time of the divide and conquer algorithms.
- For a given program (algorithm), first, we try to find the recurrence relation for the problem.
- If the recurrence is of the below form then we can directly give the answer without fully solving it.
- If the recurrence is of the form $T(n) = aT(n/b) + \Theta(n^k \log^p n)$ where $a \geq 1$, $b > 1, k \geq 0$ and p is a real number, then:

Total Three Cases:

- 1) If $a > b^k$, then $T(n) = \Theta(n^{\log_b a})$
- 2) If $a = b^k$
 - a. If $p > -1$, then $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$
 - b. If $p = -1$, then $T(n) = \Theta(n^{\log_b a} \log \log n)$
 - c. If $p < -1$, then $T(n) = \Theta(n^{\log_b a})$
- 3) If $a < b^k$
 - a. If $p \geq 0$, then $T(n) = \Theta(n^k \log^p n)$
 - b. If $p < 0$, then $T(n) = O(n^k)$

Divide and Conquer Master Theorem: Problems & Solutions

For each of the following recurrences, give an expression for the runtime $T(n)$ if the recurrence can be solved with the Master Theorem. Otherwise, indicate that the Master Theorem does not apply.

Problem-1 $T(n) = 3T(n/2) + n^2$

Solution: $T(n) = 3T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 3.a)

Problem-2 $T(n) = 4T(n/2) + n^2$

Solution: $T(n) = 4T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2 \log n)$ (Master Theorem Case 2.a)

Problem-3 $T(n) = T(n/2) + n^2$

Solution: $T(n) = T(n/2) + n^2 \Rightarrow \Theta(n^2)$ (Master Theorem Case 3.a)

Problem-4 $T(n) = 2^n T(n/2) + n^n$

Solution: $T(n) = 2^n T(n/2) + n^n \Rightarrow$ Does not apply (a is not constant)

Problem-5 $T(n) = 16T(n/4) + n$

Solution: $T(n) = 16T(n/4) + n \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 1)

Problem-6 $T(n) = 2T(n/2) + n \log n$

Solution: $T(n) = 2T(n/2) + n \log n \Rightarrow T(n) = \Theta(n \log^2 n)$ (Master Theorem Case 2.a)

Problem-7 $T(n) = 2T(n/2) + n/\log n$

Solution: $T(n) = 2T(n/2) + n/\log n \Rightarrow T(n) = \Theta(n \log \log n)$ (Master Theorem Case 2. b)

Problem-8 $T(n) = 2T(n/4) + n^{0.51}$

Solution: $T(n) = 2T(n/4) + n^{0.51} \Rightarrow T(n) = \Theta(n^{0.51})$ (Master Theorem Case 3.b)

Problem-9 $T(n) = 0.5T(n/2) + 1/n$

Solution: $T(n) = 0.5T(n/2) + 1/n \Rightarrow$ Does not apply ($a < 1$)

Problem-10 $T(n) = 6T(n/3) + n^2 \log n$

Solution: $T(n) = 6T(n/3) + n^2 \log n \Rightarrow T(n) = \Theta(n^2 \log n)$ (Master Theorem Case 3.a)

Problem-11 $T(n) = 64T(n/8) - n^2 \log n$

Solution: $T(n) = 64T(n/8) - n^2 \log n \Rightarrow$ Does not apply (function is not positive)

Problem-12 $T(n) = 7T(n/3) + n^2$

Solution: $T(n) = 7T(n/3) + n^2 \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 3.as)

Problem-13 $T(n) = 4T(n/2) + \log n$

Solution: $T(n) = 4T(n/2) + \log n \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 1)

Problem-14 $T(n) = 16T(n/4) + n!$

Solution: $T(n) = 16T(n/4) + n! \Rightarrow T(n) = \Theta(n!)$ (Master Theorem Case 3.a)

Problem-15 $T(n) = \sqrt{2}T(n/2) + \log n$

Solution: $T(n) = \sqrt{2}T(n/2) + \log n \Rightarrow T(n) = \Theta(\sqrt{n})$ (Master Theorem Case 1)

Problem-16 $T(n) = 3T(n/2) + n$

Solution: $T(n) = 3T(n/2) + n \Rightarrow T(n) = \Theta(n^{\log 3})$ (Master Theorem Case 1)

Problem-17 $T(n) = 3T(n/3) + \sqrt{n}$

Solution: $T(n) = 3T(n/3) + \sqrt{n} \Rightarrow T(n) = \Theta(n)$ (Master Theorem Case 1)

Problem-18 $T(n) = 4T(n/2) + cn$

Solution: $T(n) = 4T(n/2) + cn \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 1)

Problem-19 $T(n) = 3T(n/4) + n \log n$

Solution: $T(n) = 3T(n/4) + n \log n \Rightarrow T(n) = \Theta(n \log n)$ (Master Theorem Case 3.a)

Problem-20 $T(n) = 3T(n/3) + n/2$

Solution: $T(n) = 3T(n/3) + n/2 \Rightarrow T(n) = \Theta(n \log n)$ (Master Theorem Case 2.a)

Applications:

1. Quick sort.
2. Merge sort.
3. Majority Element.
4. Calculate pow(x,n).

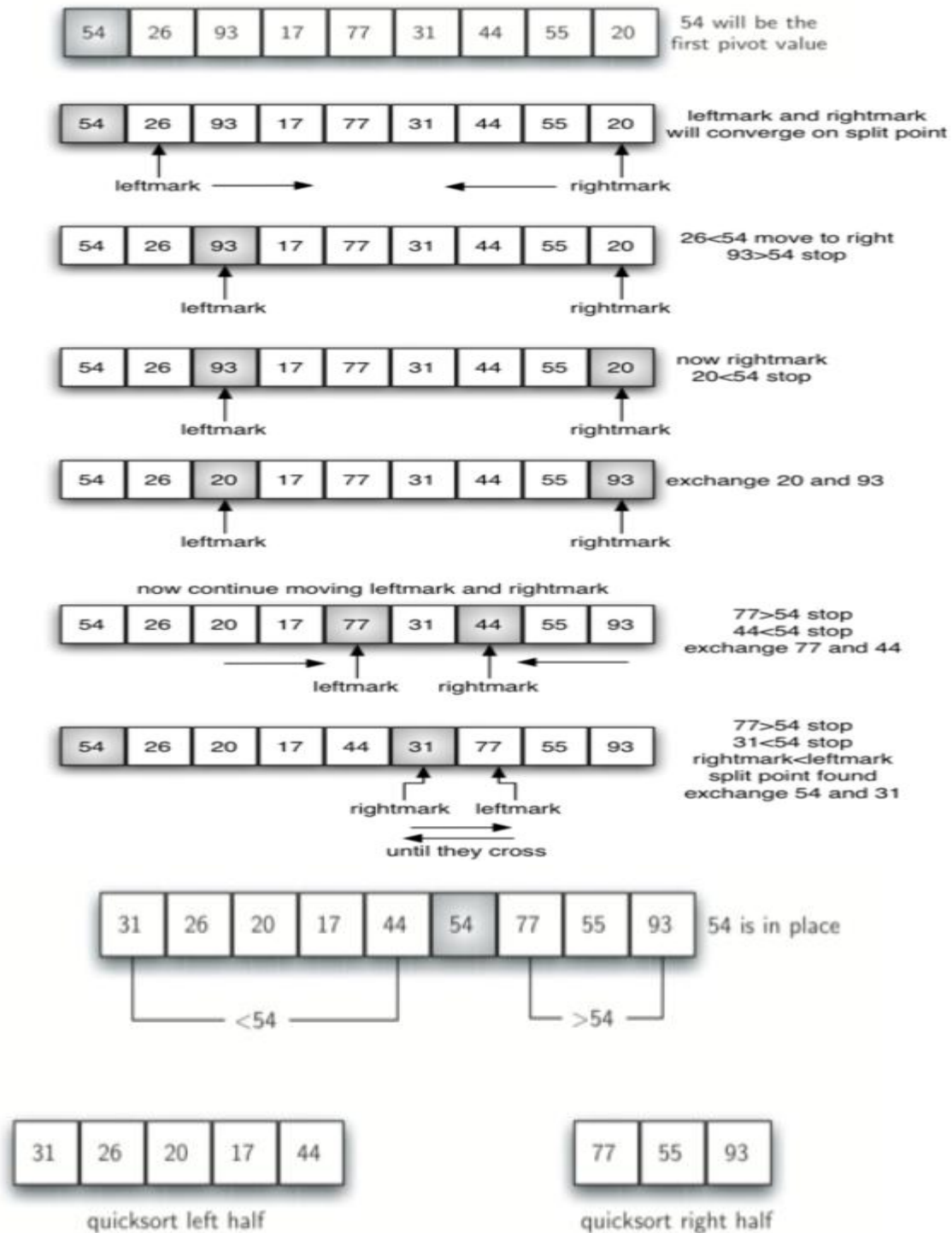
1. Quick sort.

- Quicksort is a Divide and Conquer algorithm.
- It picks an element as pivot and partitions the given array around the picked pivot.
- There are many different versions of quick Sort that pick pivot in different ways.
 - ->Always pick first element as pivot.
 - ->Always pick last element as pivot
 - ->Pick a random element as pivot.
 - ->Pick median as pivot.
- The key process in quick Sort is partition().
- Target of partitions is, given an array and an element 'x' of array as pivot,
- put 'x' at its correct position in sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x.
- All this should be done in linear time.

Divide:

- ✓ Pick any element as the **pivot**, e.g, the last element
- ✓ Partition the remaining elements into

- ✓ **FirstPart**, which contains all elements $< \text{pivot}$
- ✓ **SecondPart**, which contains all elements $> \text{pivot}$
- **Recursively sort** First Part and Second Part.
- **Combine**: no work is necessary since sorting is done in place.

Example:

Pseudo Code for Quick Sort (Recursion):

```

1  Algorithm QuickSort( $p, q$ )
2  // Sorts the elements  $a[p], \dots, a[q]$  which reside in the global
3  // array  $a[1 : n]$  into ascending order;  $a[n + 1]$  is considered to
4  // be defined and must be  $\geq$  all the elements in  $a[1 : n]$ .
5  {
6      if ( $p < q$ ) then // If there are more than one element
7      {
8          // divide  $P$  into two subproblems.
9           $j := \text{Partition}(a, p, q + 1)$ ;
10         //  $j$  is the position of the partitioning element.
11         // Solve the subproblems.
12         QuickSort( $p, j - 1$ );
13         QuickSort( $j + 1, q$ );
14         // There is no need for combining solutions.
15     }
16 }

1  Algorithm Partition( $a, m, p$ )
2  // Within  $a[m], a[m + 1], \dots, a[p - 1]$  the elements are
3  // rearranged in such a manner that if initially  $t = a[m]$ ,
4  // then after completion  $a[q] = t$  for some  $q$  between  $m$ 
5  // and  $p - 1$ ,  $a[k] \leq t$  for  $m \leq k < q$ , and  $a[k] \geq t$ 
6  // for  $q < k < p$ .  $q$  is returned. Set  $a[p] = \infty$ .
7  {
8       $v := a[m]$ ;  $i := m$ ;  $j := p$ ;
9      repeat
10     {
11         repeat
12              $i := i + 1$ ;
13         until ( $a[i] \geq v$ );
14
15         repeat
16              $j := j - 1$ ;
17         until ( $a[j] \leq v$ );
18
19         if ( $i < j$ ) then Interchange( $a, i, j$ );
20     } until ( $i \geq j$ );
21
22      $a[m] := a[j]$ ;  $a[j] := v$ ; return  $j$ ;
23 }

1  Algorithm Interchange( $a, i, j$ )
2  // Exchange  $a[i]$  with  $a[j]$ .
3  {
4       $p := a[i]$ ;
5       $a[i] := a[j]$ ;  $a[j] := p$ ;
6  }

```

Pseudo Code for Quick Sort (Iterative):

```

1  Algorithm QuickSort2( $p, q$ )
2  // Sorts the elements in  $a[p : q]$ .
3  {
4      // stack is a stack of size  $2 \log(n)$ .
5      repeat
6      {
7          while ( $p < q$ ) do
8          {
9               $j := \text{Partition}(a, p, q + 1)$ ;
10             if  $((j - p) < (q - j))$  then
11             {
12                  $\text{Add}(j + 1)$ ; // Add  $j + 1$  to stack.
13                  $\text{Add}(q)$ ;  $q := j - 1$ ; // Add  $q$  to stack
14             }
15             else
16             {
17                  $\text{Add}(p)$ ; // Add  $p$  to stack.
18                  $\text{Add}(j - 1)$ ;  $p := j + 1$ ; // Add  $j - 1$  to stack
19             }
20             } // Sort the smaller subfile.
21             if stack is empty then return;
22              $\text{Delete}(q)$ ;  $\text{Delete}(p)$ ; // Delete  $q$  and  $p$  from stack.
23         } until (false);
24     }

```

Pseudo Code for Randomized Quick Sort Algorithm:

```

1  Algorithm RQuickSort( $p, q$ )
2  // Sorts the elements  $a[p], \dots, a[q]$  which reside in the global
3  // array  $a[1 : n]$  into ascending order.  $a[n + 1]$  is considered to
4  // be defined and must be  $\geq$  all the elements in  $a[1 : n]$ .
5  {
6      if ( $p < q$ ) then
7      {
8          if  $((q - p) > 5)$  then
9               $\text{Interchange}(a, \text{Random}() \bmod (q - p + 1) + p, p)$ ;
10              $j := \text{Partition}(a, p, q + 1)$ ;
11             //  $j$  is the position of the partitioning element.
12             RQuickSort( $p, j - 1$ );
13             RQuickSort( $j + 1, q$ );
14         }
15     }

```

Time complexity analysis:

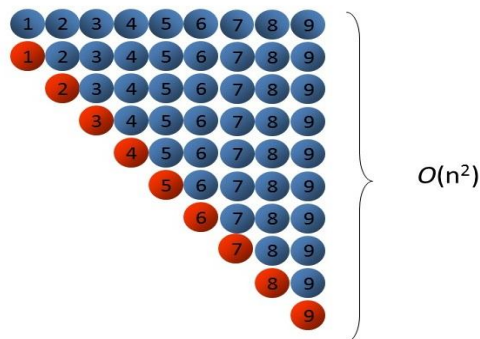
- The time required to sort n elements using quicksort involves 3 components.
 - Time required for partitioning the array, which is *roughly proportional to n* .
 - Time required for sorting lower subarray.
 - Time required for sorting upper subarray.
- Assume that there are k elements in the lower subarray.
- Therefore,

$$T(n) = C_1 \quad n=1, \text{ } C_1 \text{ is a constant}$$

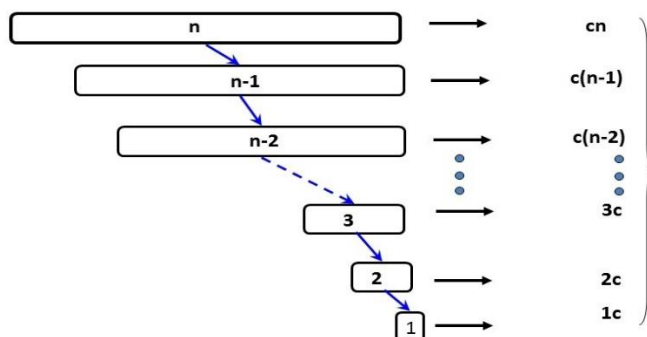
$$T(n) = 2T(n/2) + c n \quad n > 1$$

A worst/bad case

It occurs if the list is already in sorted order



Worst/bad Case



contd...

- In the worst case, the array is always partitioned into two subarrays in which one of them is always empty. Thus , for the worst case analysis,

$$\begin{aligned}
 T(n) &= \begin{cases} T(n-1) + c_2 n & n > 1, c_2 \text{ is a constant} \\ \end{cases} \\
 &= T(n-1) + c_2 n \\
 &= T(n-2) + c_2 (n-1) + c_2 n \\
 &= T(n-3) + c_2 (n-2) + c_2 (n-1) + c_2 n \\
 &\quad \dots \\
 &\quad \dots \\
 &= n(n+1)/2 = (n^2+n)/2 = \mathbf{O(n^2)}
 \end{aligned}$$

Time complexities:

- Most of the work done in partitioning
- **Best case** takes $O(n \log(n))$ time
- **Average case** takes $O(n \log(n))$ time
- **Worst case** takes $O(n^2)$ time

Quick sort using recursion in java: Quicksort_Recursive.java

```

import java.util.*;
class QuickSort
{
    //selects last element as pivot, pi using which array is partitioned.
    int partition(int intArray[], int low, int high)
    {
        int pi = intArray[high];
        int i = (low-1); // smaller element index
        for (int j=low; j<high; j++)
        {
            // check if current element is less than or equal to pi
            if (intArray[j] <= pi)
            {
                i++;
                // swap intArray[i] and intArray[j]
                int temp = intArray[i];
                intArray[i] = intArray[j];
                intArray[j] = temp;
            }
        }
    }
}

```

```
    }
}
// swap intArray[i+1] and intArray[high] (or pi)
int temp = intArray[i+1];
intArray[i+1] = intArray[high];
intArray[high] = temp;

return i+1;
}
//routine to sort the array partitions recursively
void quick_sort(int intArray[], int low, int high)
{
    if (low < high) {
        //partition the array around pi=>partitioning index and return pi
        int pi = partition(intArray, low, high);

        // sort each partition recursively
        quick_sort(intArray, low, pi-1);
        quick_sort(intArray, pi+1, high);
    }
}
}
class Quicksort_Recursive
{
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);
        System.out.println("enter array size");

        int n = sc.nextInt();
        int a[]=new int[n];

        System.out.println("enter the elements of array ");

        for(int i=0;i<n;i++)
        {
            a[i] =sc.nextInt();
        }

        //print the original array

        System.out.println ("Original Array: " + Arrays.toString(a));

        //call quick_sort routine using QuickSort object
```

```
QuickSort obj = new QuickSort();

obj.quick_sort(a, 0, n-1);

//print the sorted array
System.out.println("Sorted Array: " + Arrays.toString(a));
}
```

Output:

enter array size

5

enter the elements of array

10

9

5

78

14

Original Array: [10, 9, 5, 78, 14]

Sorted Array: [5, 9, 10, 14, 78]

Iterative Quicksort

In iterative quicksort, we use the auxiliary stack to place intermediate parameters instead of using recursion and sort partitions.

Java program implements iterative quicksort: Quicksort_iterative.java

```
import java.util.*;

class Quicksort_iterative
{
    //partitions the array around pivot=> last element
    static int partition(int numArray[], int low, int high)
    {
        int pivot = numArray[high];
        // smaller element index
        int i = (low - 1);
        for (int j = low; j <= high - 1; j++) {
            // check if current element is less than or equal to pivot
            if (numArray[j] <= pivot) {
                i++;
                // swap the elements
                int temp = numArray[i];
                numArray[i] = numArray[j];
            }
        }
    }
}
```

```
        numArray[j] = temp;
    }
}
// swap numArray[i+1] and numArray[high] (or pivot)
int temp = numArray[i + 1];
numArray[i + 1] = numArray[high];
numArray[high] = temp;
return i + 1;
}
//sort the array using quickSort
static void quickSort(int numArray[], int low, int high)
{
    //auxillary stack
    int[] intStack = new int[high - low + 1];

    // top of stack initialized to -1
    int top = -1;

    // push initial values of low and high to stack
    intStack[++top] = low;
    intStack[++top] = high;
    // Keep popping from stack while is not empty
    while (top >= 0) {
        // Pop h and l
        high = intStack[top--];
        low = intStack[top--];
        // Set pivot element at its correct position
        // in sorted array
        int pivot = partition(numArray, low, high);

        // If there are elements on left side of pivot,
        // then push left side to stack
        if (pivot - 1 > low) {
            intStack[++top] = low;
            intStack[++top] = pivot - 1;
        }
        // If there are elements on right side of pivot,
        // then push right side to stack
        if (pivot + 1 < high) {
            intStack[++top] = pivot + 1;
            intStack[++top] = high;
        }
    }
}
```

```
    }  
    }  
}  
public static void main(String args[])  
{  
  
    Scanner sc=new Scanner(System.in);  
    System.out.println("enter array size");  
    int n = sc.nextInt();  
    int a[]=new int[n];  
    System.out.println("enter the elements of array ");  
    for(int i=0;i<n;i++)  
    {  
        a[i] =sc.nextInt();  
    }  
    System.out.println("Original Array:" + Arrays.toString(a));  
    // call quickSort routine to sort the array  
    quickSort(a, 0, n - 1);  
    //print the sorted array  
    System.out.println("\nSorted Array:" + Arrays.toString(a));  
}  
}
```

Output:

enter array size

5

enter the elements of array

12

32

41

56

20

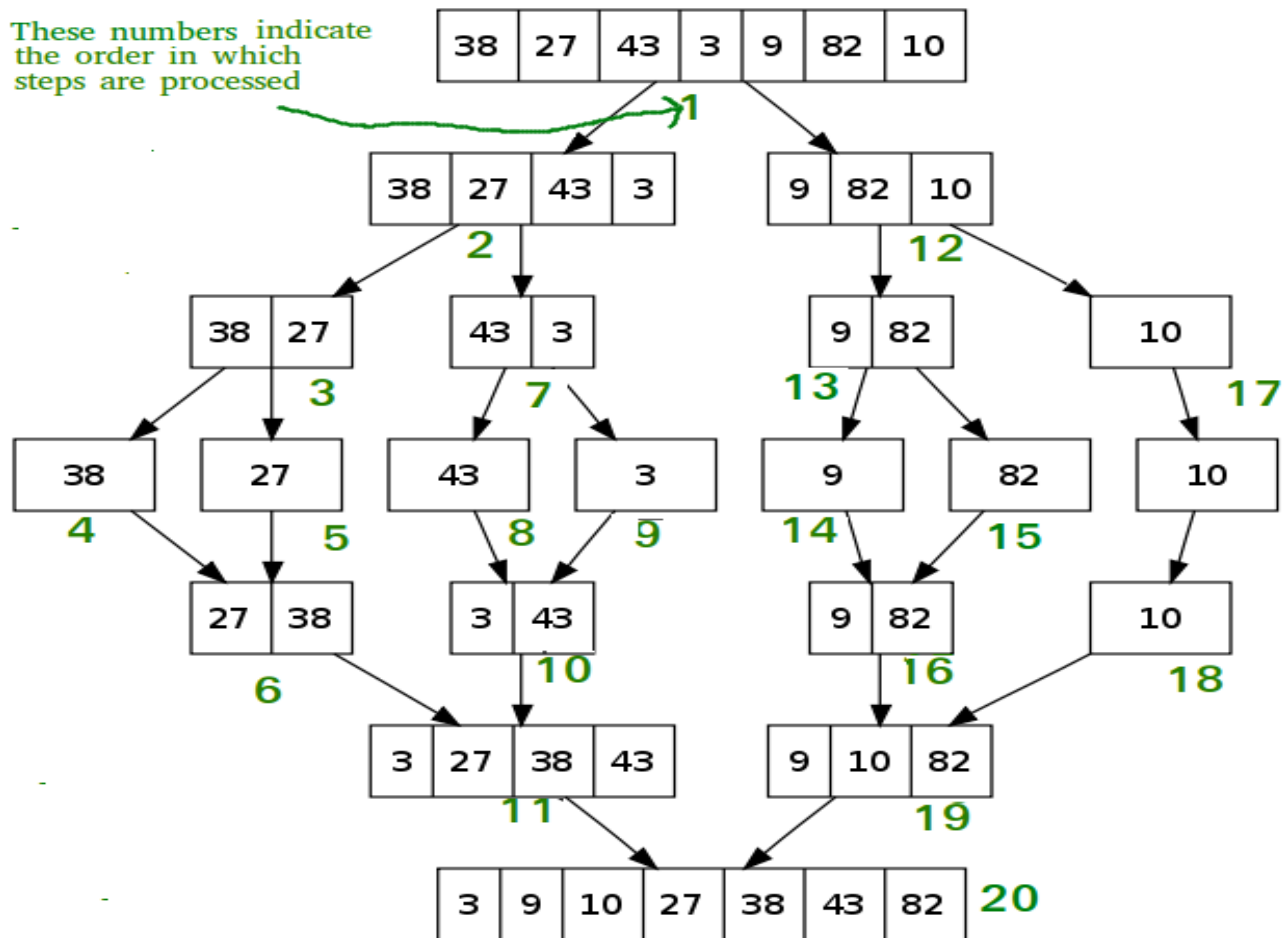
Original Array:[12, 32, 41, 56, 20]

Sorted Array:[12, 20, 32, 41, 56]

2. Merge Sort:

- The Merge sort algorithm can be used to sort a collection of objects.
- Merge sort is also called as divide and conquer algorithm.
- Base Case, solve the problem directly if it is small enough(only one element).
- Divide the problem into two or more similar and smaller subproblems.
- Recursively solve the subproblems.
- Combine solutions to the subproblems

Merge Sort Example



Pseudo code:

```

1  Algorithm MergeSort(low, high)
2  // a[low : high] is a global array to be sorted.
3  // Small(P) is true if there is only one element
4  // to sort. In this case the list is already sorted.
5  {
6      if (low < high) then // If there are more than one element
7      {
8          // Divide P into subproblems.
9          // Find where to split the set.
10         mid :=  $\lfloor (low + high) / 2 \rfloor$ ;
11         // Solve the subproblems.
12         MergeSort(low, mid);
13         MergeSort(mid + 1, high);
14         // Combine the solutions.
15         Merge(low, mid, high);
16     }
17 }

```

```

1  Algorithm Merge(low, mid, high)
2  // a[low : high] is a global array containing two sorted
3  // subsets in a[low : mid] and in a[mid + 1 : high]. The goal
4  // is to merge these two sets into a single set residing
5  // in a[low : high]. b[ ] is an auxiliary global array.
6  {
7      h := low; i := low; j := mid + 1;
8      while ((h ≤ mid) and (j ≤ high)) do
9      {
10         if (a[h] ≤ a[j]) then
11         {
12             b[i] := a[h]; h := h + 1;
13         }
14         else
15         {
16             b[i] := a[j]; j := j + 1;
17         }
18         i := i + 1;
19     }
20     if (h > mid) then
21         for k := j to high do
22         {
23             b[i] := a[k]; i := i + 1;
24         }
25     else
26         for k := h to mid do
27         {
28             b[i] := a[k]; i := i + 1;
29         }
30     for k := low to high do a[k] := b[k];
31 }

```

Calculating Time complexity Using Substitute Method:

If the time for the merging operation is proportional to n , then the computing time for merge sort is described by the recurrence relation

$$T(n) = \begin{cases} a & n = 1, a \text{ a constant} \\ 2T(n/2) + cn & n > 1, c \text{ a constant} \end{cases}$$

When n is a power of 2, $n = 2^k$, we can solve this equation by successive substitutions:

$$\begin{aligned} T(n) &= 2(2T(n/4) + cn/2) + cn \\ &= 4T(n/4) + 2cn \\ &= 4(2T(n/8) + cn/4) + 2cn \\ &\vdots \\ &= 2^k T(1) + kcn \\ &= an + cn \log n \end{aligned}$$

It is easy to see that if $2^k < n \leq 2^{k+1}$, then $T(n) \leq T(2^{k+1})$. Therefore

$$T(n) = O(n \log n)$$

Time complexities: Most of the work done in combining the solutions.

- **Best case :** $O(n \log(n))$ time
- **Average case :** $O(n \log(n))$ time
- **Worst case :** $O(n \log(n))$ time

Applications for merge sort in real time:

- Merge sort is useful for sorting linked lists.
- Inversion count problem
- Used in external sorting- in tape drivers
- e commerce applications

Merge sort Using Recursion in java:**Mergesort_recursive.java**

```
import java.util.Scanner;

public class Mergesort_recursive
{
    public static void main(String a[])
    {

        int temp;
        Scanner sc=new Scanner (System.in);
        System.out.println("Enter array size");
        int n = sc.nextInt();
        int [] list=new int[n];

        System.out.println("Enter numbers ");

        for (int i=0 ; i<n; i++)
        {
            int number = sc.nextInt();
            list[i]=number;
        }
        System.out.println("List before sorting \n");
        for(int i = 0; i < list.length; i++)
            System.out.println( list[i]+" ");
        mergeSort(list,0, list.length-1);
        System.out.print("List after sorting \n");
        for(int i = 0; i < list.length; i++)
        {
            System.out.print(list[i]+" ");
        }
        public static void mergeSort(int list[],int low, int high){
            if (low >= high)
            {
                return;
            }
            int middle = (low + high) / 2;

            mergeSort(list, low, middle);

            mergeSort(list, middle + 1, high);

            merge(list, low,middle,high);
        }
        private static void merge(int list[], int low, int middle, int high)
```

```
{
    int IstList_end= middle;

    int IIndList_start = middle + 1;

    int l=low;
    while ((l <= IstList_end) && (IIndList_start <= high))
    {
        if (list[l] < list[IIndList_start])
        {
            low++;
        } else {
            int temp = list[IIndList_start];
            for (int j = IIndList_start-1; j >= low; j--) {
                list[j+1] = list[j];
            }
            list[l] = temp;
            low++;
            IstList_end++;
            IIndList_start++;
        }
    }
}
```

Output:

Enter array size

5

Enter numbers

9

8

10

7

4

List before sorting

9 8 10 7 4

List after sorting

4 7 8 9 10

Merge sort using iteration in java: Mergesort_iterative.java

```
import java.lang.Math.*;
import java.util.*;

class Mergesort_iterative
{
    /* Iterative mergesort function to sort arr[0...n-1] */
    static void mergeSort(int arr[], int n)
    {
        int curr_size;

        // For picking starting index of
        // left subarray to be merged
        int left_start;
        // Merge subarrays in bottom up
        // manner. First merge subarrays
        // of size 1 to create sorted
        // subarrays of size 2, then merge
        // subarrays of size 2 to create
        // sorted subarrays of size 4, and
        // so on.
        for (curr_size = 1; curr_size <= n-1;
            curr_size = 2*curr_size)
        {

            // Pick starting point of different
            // subarrays of current size
            for (left_start = 0; left_start < n-1;
                left_start += 2*curr_size)
            {
                // Find ending point of left
                // subarray. mid+1 is starting
                // point of right
                int mid = Math.min(left_start + curr_size - 1, n-1);

                int right_end = Math.min(left_start
                    + 2*curr_size - 1, n-1);

                // Merge Subarrays arr[left_start...mid]
                // & arr[mid+1...right_end]
                merge(arr, left_start, mid, right_end);
            }
        }
    }

    /* Function to merge the two halves arr[l..m] and
    arr[m+1..r] of array arr[] */
}
```

```
static void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    /* create temp arrays */
    int L[] = new int[n1];
    int R[] = new int[n2];
    /* Copy data to temp arrays L[]
    and R[] */
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
    /* Merge the temp arrays back into
    arr[l..r]*/
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    /* Copy the remaining elements of
    L[], if there are any */
    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }
    /* Copy the remaining elements of R[], if there are any */
    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

```
    }  
    /* Function to print an array */  
    static void printArray(int A[], int size)  
    {  
        int i;  
        for (i=0; i < size; i++)  
            System.out.printf("%d ", A[i]);  
        System.out.printf("\n");  
    }  
  
    /* Driver program to test above functions */  
    public static void main(String[] args)  
    {  
        Scanner sc=new Scanner (System.in);  
        System.out.println("Enter array size");  
        int n = sc.nextInt();  
        int [] arr=new int[n];  
        System.out.println("Enter numbers ");  
        for (int i=0 ; i<n; i++)  
        {  
            int number = sc.nextInt();  
            arr[i]=number;  
        }  
        System.out.printf("Given array is \n");  
        printArray(arr, n);  
        mergeSort(arr, n);  
        System.out.printf("\nSorted array is \n");  
        printArray(arr, n);  
    }  
}
```

Input:

Enter array size

5

Enter numbers

12 13 10 9 8

Output:

Given array is

12 13 10 9 8

Sorted array is

8 9 10 12 13

3. Majority Element:

Given an integer array nums of size n, determine whether the array contains a majority element.

A majority element is an element that appears more than $n/2$ times in the array.

implement a Divide and Conquer algorithm to find the majority element. If no majority element exists, return -1.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Explanation: Element 2 appears 4 times. $\lceil 7/2 \rceil = 3$ Since $4 > 3$, 2 is the majority element.

Example 3:

Input: nums = [1,2,2,1,3,1]

Output: -1

Naïve Approach $O(n^2)$ Time Complexity:

```
import java.util.Scanner;

class Majority {

    static int findMajority(int arr[], int n)
    {
        for (int i = 0; i < n; i++) {

            int count = 0;

            for (int j = 0; j < n; j++) {

                if (arr[i] == arr[j])
                    count++;

            }

            if (count > n / 2)
                return arr[i];

        }
        return -1;
    }

    public static void main(String args[]) {
        int arr[] = {2,2,1,1,1,2,2}
        System.out.println(findMajority(arr, n));
        sc.close();
    }
}
```

Divide & Conquer Approach:

1. **Divide the array** into two equal halves using the middle index.
2. **Recursively** find the majority element in the left half and the right half.
3. If both recursive calls return the **same element**, return it as the majority candidate.
4. Otherwise, **count the frequency** of both candidates in the current subarray.
5. If the frequency of the left candidate is **greater than half** of the current subarray size, return the left candidate.
6. Else if the frequency of the right candidate is **greater than half** of the current subarray size, return the right candidate.
7. If neither candidate satisfies the majority condition, return -1.

Algorithm:

1. If the subarray contains only one element, return that element.
2. Divide the array into two halves.
3. Recursively find the majority element in the left half.
4. Recursively find the majority element in the right half.
5. If both majority elements are the same, return that element.
6. Count the occurrences of both candidates in the current subarray.
7. Return the candidate whose frequency is greater than half of the current subarray size.
8. If no candidate satisfies the condition, return -1.

Complexity Analysis:

- **Time Complexity:** $O(n \log n)$
- **Space Complexity:** $O(\log n)$ (Recursion Stack)

Java program for Majority using D&C Recursion:**Majority.java**

```
import java.util.Scanner;

class Majority {

    // Function to find the majority element using Divide & Conquer
    static int getMajority(int arr[], int left, int right) {

        // Base Case
        if (left == right)
            return arr[left];

        // Divide
        int mid = (left + right) / 2;

        // Conquer
        int leftMajority = getMajority(arr, left, mid);
        int rightMajority = getMajority(arr, mid + 1, right);
```

```
// If both halves return the same majority element
if (leftMajority == rightMajority)
    return leftMajority;

// Count the frequency of left and right majority candidates
int leftCount = countFrequency(arr, left, right, leftMajority);
int rightCount = countFrequency(arr, left, right, rightMajority);

// Return the element that is the majority in the current subarray
if (leftCount > (right - left + 1) / 2)
    return leftMajority;
else if (rightCount > (right - left + 1) / 2)
    return rightMajority;
else
    return -1;
}

// Function to count the frequency of an element in a given range
static int countFrequency(int arr[], int left, int right, int element)
{
    int count = 0;
    for (int i = left; i <= right; i++) {
        if (arr[i] == element)
            count++;
    }
    return count;
}

public static void main(String args[])
{
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();

    int majority = getMajority(arr, 0, n - 1);

    // Print majority element or -1 if it does not exist
    System.out.println(majority);

    sc.close();
}
}
```

Example-1:**Input=7**

2 2 1 1 1 2 2

Output=2

4. Calculate pow(x,n):

- ✓ Given two integers, x and n , where x and n are +ve are -ve numbers, efficiently compute the power function $\text{pow}(x, n)$.

For example,

1. $\text{pow}(-2, 10) = 1024$
2. $\text{pow}(-3, 4) = 81$
3. $\text{pow}(5, 0) = 1$
4. $\text{pow}(-2, -3) = -0.125$

Naïve Approach: with $O(n)$ Time complexity (Not D&C just Naïve Approach)

Procedure: A simple solution to calculate $\text{pow}(x, n)$ would multiply x exactly n times. We can do that by using a simple for loop.

```
import java.util.Scanner;

public class CalculatePowerIterative
{
    public static long power(int x, int n)
    {
        long result = 1;
        for (int i = 1; i <= n; i++)
        {
            result = result * x;
        }
        return result;
    }

    public static void main(String[] args)
    {
        int x = -2;
        int n = 10;
        System.out.println("pow(" + x + ", " + n + ") = " + power(x, n));
    }
}
```

Divide & Conquer Approach:

1. If the exponent n is 0, return 1 (base case).
2. Recursively compute $\text{power}(x, n/2)$ and store the result in a temporary variable.
3. If n is **even**, return $\text{temp} \times \text{temp}$.
4. If n is **odd** and positive, return $x \times \text{temp} \times \text{temp}$.
5. If n is **odd** and negative, return $(\text{temp} \times \text{temp}) / x$.
6. Since the exponent is divided by 2 in every recursive call, the recursion depth is $O(\log n)$.

Algorithm

Algorithm: Power(x, n)

Input: Base x, Exponent n

Output: x^n

Step 1: If $n == 0$
return 1

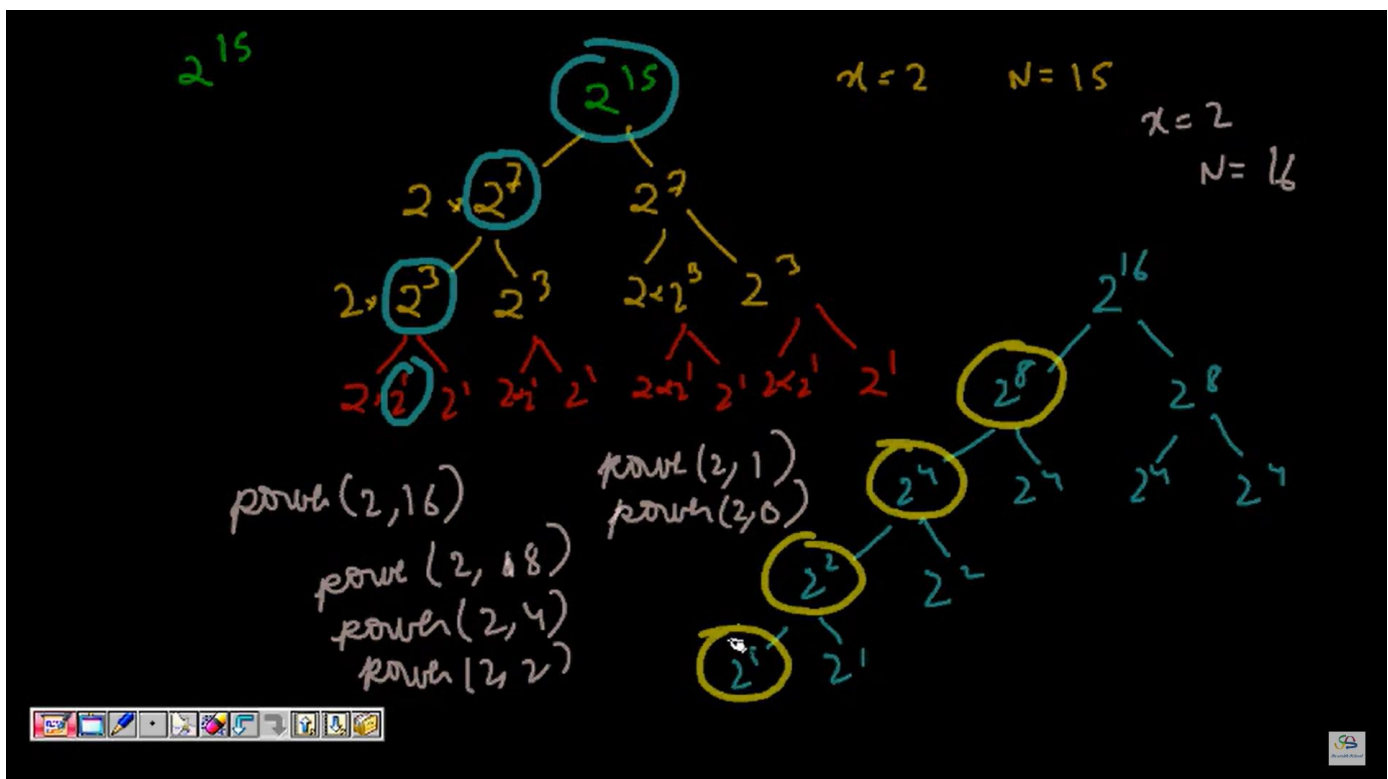
Step 2: $\text{temp} \leftarrow \text{Power}(x, n/2)$

Step 3: If n is even
return $\text{temp} \times \text{temp}$

Step 4: Else
If $n > 0$
return $x \times \text{temp} \times \text{temp}$
Else
return $(\text{temp} \times \text{temp}) / x$

Step 5: End

Pictorial Representation: if $x=2$ and $n=15$ & If $x=2$ and $N=16$



Computing Time Complexity: Let $T(n)$ be the time required to compute $\text{power}(x, n)$.

At each recursive call:

- The exponent is divided by 2.
- Only **one recursive call** is made.
- The remaining work (multiplications and comparisons) takes **constant time, $O(1)$** .

Therefore, the recurrence relation is: $T(n) = T(n/2) + O(1)$

Time Complexity: $O(\log n)$

Space Complexity: $O(\log n)$ (due to the recursion stack)

Java program for calculate Power(x,n) using D&C:

Calulatepower_Recursion.java

```
import java.util.*;

class Calulatepower_Recursion
{
    static double power(double x, int y)
    {
        double temp;
        if( y == 0)
            return 1;
        temp = power(x, y/2);
        if (y%2 == 0)
            return temp*temp;
        else
        {
            if(y > 0)
                return x * temp * temp;
            else
                return (temp * temp) / x;
        }
    }
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.println("enter x");
        double x = sc.nextDouble();
        System.out.println("enter n");
        int n = sc.nextInt();
        System.out.println( power(x, n));
    }
}
```

Example-1:**input=**

enter x

2

enter n

3

output=

8.0

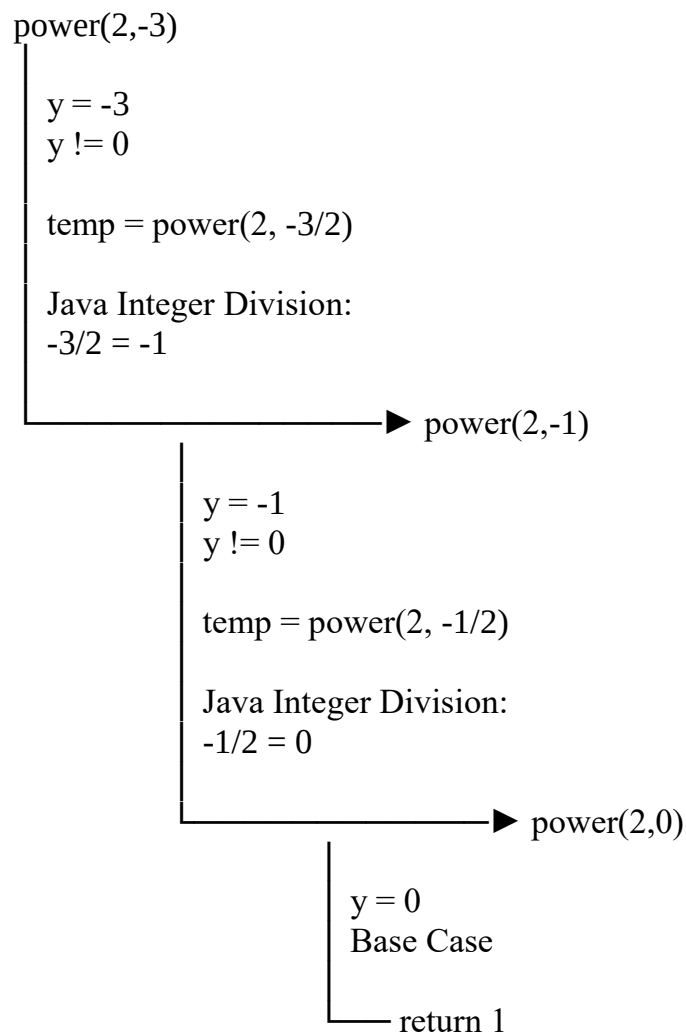
Example-2:**input =**

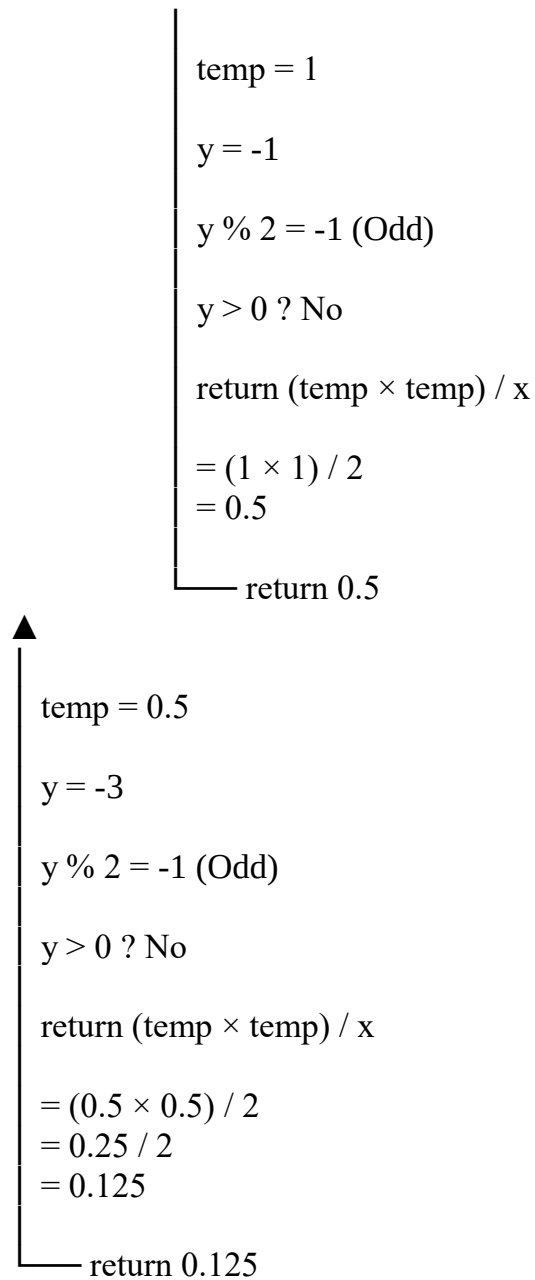
enter x

-2

enter n

-3

output = -0.125**Tracing for negative power: Example: Power(2,-3)**

**Call Sequence:**

```

power(2,-3)
↓
temp = power(2,-3/2)
↓
temp = power(2,-1)
↓
temp = power(2,-1/2)
↓
temp = power(2,0)
↓
  
```

return 1

↓

return 0.5

↓

return 0.125